

# Serial Communications

---

Chao-Chin Wu

Reference: M. Margolis, et al., Arduino Cookbook, 3<sup>rd</sup> Edition,  
Oreilly, 2020

# Introduction

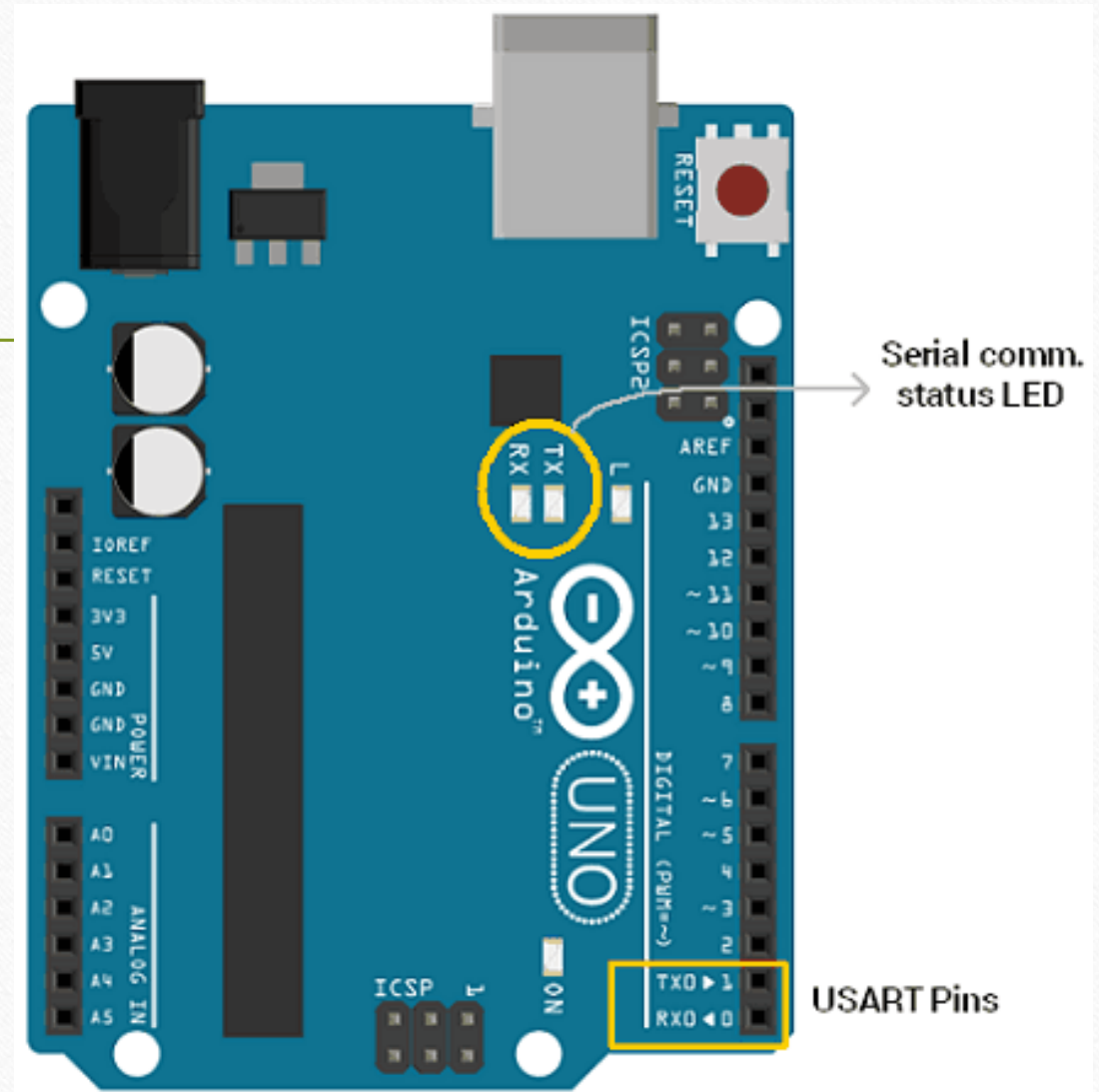
---

- Serial communication
  - an easy and flexible way for your Arduino board to interact with your computer and other devices
  - connect the Arduino USB serial port to your computer to upload sketches



# Serial hardware

- Arduino
  - 0 volts to represent a bit value of 0
  - 5 volts (or 3.3 volts) to represent a bit value of 1



- 
- Some serial devices use the RS-232 standard for serial connection
    - usually a nine-pin connector
    - an adapter is required to connect them with the Arduino
    - RS-232 use voltage levels that will damage Arduino digital pins
    - need to obtain an RS-232 to TTL adaptor to use it.



- 
- An Arduino Uno has a single hardware serial port
    - serial communication is also possible using software libraries to emulate additional ports
    - software serial is not as fast or efficient as hardware serial.



- 
- On boards other than Uno, you might need the following code to ensure that you can see serial output that you send in `setup()`

```
while (! Serial) {  
    ; // wait for serial port to connect  
}
```

# Emulate Serial Hardware with Digital Pins

---

- Serial libraries simplify the use of the serial ports by insulating you from hardware complexity.

# Message Protocols

---

- Header
  - one or more special characters that identify the start of the message
- Footer
  - .... the end of the message
- Message
  - text
  - binary



# Arduino Serial Notes

---

- Serial.flush
  - waits for all outgoing data to be sent rather than discarding received data (old version)
  - to discard received data
    - `while (Serial.read() >= 0) ; //flush the received buffer`
- Serial.write and Serial.print do not block.
- If you want to wait until all characters are sent, call Serial.flush immediately following Serial.write or Serial.print

- 
- `Serial.print` returns the number of characters printed.
  - `Serial.peek` lets you “peek” the next character in the receive buffer.
    - the character is not removed from the buffer

# Sending Data to More than One Serial Device

---

- Arduino Uno has only one hardware serial port.
  - need to create an emulated or “soft” serial port using the SoftwareSerial library.
  - Select two available digital pins, one each for transmit and receive
  - connect your serial device to them
  - Tx  $\longleftrightarrow$  Rx



```
/*  
  SoftwareSerialOutput sketch  
  Output data to a software serial port  
*/
```

```
#include <SoftwareSerial.h>
```

---

```
const int rxpin = 2;      // pin used to receive (not used in this version)  
const int txpin = 3;      // pin used to send to LCD  
SoftwareSerial serial_lcd(rxpin, txpin); // new serial port on pins 2 and 3
```

```
void setup()  
{  
  Serial.begin(9600); // 9600 baud for the built-in serial port  
  serial_lcd.begin(9600); // initialize the software serial port also for 9600  
}
```

```
int number = 0;
```

---

```
void loop()
{
  serial_lcd.print("Number: "); // send text to the LCD
  serial_lcd.println(number);   // print the number on the LCD
  Serial.print("Number: ");
  Serial.println(number);       // print the number on the PC console

  delay(500); // delay half second between numbers
  number++;   // to the next number
}
```

- 
- On the Arduino Uno, the built-in hardware serial port is connected to the USB serial connection.
    - wired to pins 0 (receive) and 1 (transmit)
    - allowing to connect a device such as an LCD serial display to the Arduino.
    - The characters you transmit over the Serial object are display on the LCD
  - Note: Although you can use a separate power supply for the serial device, you must connect the Arduino's ground pin to the device's pin.
    - giving the Arduino and the serial device a common ground.